

- Modified instruction memory must have the coherence PMA.

Other well-defined code-modification strategies exist, but these rules provide a safe harbor.

Note that the software modifying the code need not know the value of **M**. Because **ILEN** must be at least the width of the instruction being modified, a lower bound on **M** can be inferred from the instruction's width and **XLEN**.

Memory protection and executability PMAs are applied only to bytes that are not discarded by this algorithm.



For example, if **M=8**, **N=0**, and the PMP granularity is 4 bytes, then it is valid to fetch a 4-byte instruction at **pc**, even if fetching from **pc + 4** would have been disallowed by PMP.

For simplicity, implementations are likely to choose a PMP granularity no smaller than **M**.

4.10. ziccid Extension for Instruction/Data Coherence and Consistency

The **Ziccid** extension mandates more stringent requirements for consistency between instruction fetches and other memory accesses than those imposed by the base ISA. As described in detail in the next section, **Ziccid** guarantees that a hart's instruction fetches appear to occur in program order with respect to each other, and that stores eventually become visible to all harts' instruction fetches.



The primary intent of the **Ziccid** extension is to accelerate JIT compilation in multiprocessor systems.

In straightforward implementations, maintaining coherence between instruction caches and the data-memory system suffices to satisfy this extension's eventuality property. The **Ziccid** extension can be viewed as a means to codify the concept of instruction-cache coherence.

The in-order fetch property, expected by some JIT compilers, is also provided naturally by most straightforward microarchitectures. An example of a technique that might violate this property, however, is an instruction buffer that is populated out of program order, e.g., while fetching down the predicted path following an instruction-cache miss. Example solutions include buffering instructions only in program order, or keeping the buffer coherent by making the coherent instruction cache inclusive of the buffer.

Under **Ziccid**, stores eventually become visible to instruction fetches, even without executing a **FENCE.I** instruction. As a consequence of this requirement, the consumer thread in the following litmus test is guaranteed to terminate:



Producer:

```
la t0, patch_me
li t1, 0x4585
sh t1, (t0)    # patch_me := c.li a1, 1
```

Consumer:

```
patch_me:
c.li a1, 0
beqz a1, patch_me
```

As is the case without the **Ziccid** extension, software can make a store from the current hart visible to the hart's instruction fetches immediately by executing a **FENCE.I** instruction. Software can make a store from the current hart visible to another hart's instruction fetches by executing a **FENCE.W,O** instruction, then sending an interprocessor interrupt to the remote hart, requesting that it execute a **FENCE.I** instruction. Alternatively, the current hart can

execute a FENCE W,W instruction, then write a flag in memory; upon observing the flag write, the remote hart executes a FENCE.I instruction. In neither case is a data FENCE required on the remote hart.

The **Ziccid** extension depends on the **Ziccif** extension.



Although instruction/data consistency and instruction-fetch atomicity are conceptually independent properties, the former is much more useful in conjunction with the latter.

4.10.1. Operational Model of the **Ziccid** Extension

An implementation with the **Ziccid** extension behaves in a manner consistent with the following operational model. This model only applies to instruction fetches to memory regions with the cacheability and coherence PMAs; instruction fetches to other memory regions proceed as though the **Ziccid** extension were not implemented.



Informally, instructions in incoherent and/or uncacheable memory regions may be incoherently cached; this cache is flushed upon execution of a FENCE.I instruction.

Harts have two ordered operational stages: instruction fetch and execution. Each hart has an instruction buffer of bounded capacity. Instructions are fetched from memory in program order, then inserted into the instruction buffer in program order.

An instruction fetch is a memory read operation that appears in the global memory order, in program order with respect to other instruction fetches from the same hart. For each instruction byte fetched from memory, memory supplies the value written by the most recent store that precedes the read in global memory order.



This property is distinct from than the load-value axiom, in that fetches do not observe the local hart's stores before other harts' fetches can observe them.

Instruction fetches appear in the global memory order before any explicit memory accesses to which they give rise.



Absent the execution of a FENCE.I instruction, there is no requirement that younger instruction fetches appear in the global memory order later than older explicit memory accesses. For example, if a program consists of two store instructions X and Y to distinct addresses AX and AY, then the following global orders are valid:

- *fetch X, fetch Y, store AX, store AY*
- *fetch X, fetch Y, store AY, store AX*
- *fetch X, store AX, fetch Y, store AY*

After some amount of time, instructions are dequeued in program order from the instruction buffer and executed.



Therefore, absent the execution of a FENCE.I instruction, stores do not necessarily become visible immediately to subsequent instruction fetches, though they do eventually become visible to subsequent instruction fetches.

Instructions are executed in program order, but instructions' explicit memory accesses may be reordered in any manner consistent with RVWMO.



Executing a FENCE w, w instruction between two store instructions, for example, suffices to

guarantee that the second store will not become visible to any instruction fetch before the first store does.

The FENCE.I instruction performs the following operations sequentially:

- All of the hart's older explicit memory accesses become globally visible.
- The instruction buffer is flushed.



The FENCE.I instruction in Ziccid is specified such that software written for non-Ziccid machines is compatible with Ziccid machines. For cacheable and coherent memory regions, the combination of making older stores globally visible and flushing the instruction buffer on FENCE.I suffices to implement FENCE.I's pre-Ziccid semantics for those regions.

4.11. Ziccrse Extension for Main Memory Reservability

If the Ziccrse extension is implemented, then main memory regions with both the coherence and cacheability PMAs must support the RsrvEventual PMA.

4.12. Ziccamoa Extension for Main Memory Atomics

If the Ziccamoa extension is implemented, then main memory regions with both the coherence and cacheability PMAs must provide AMOArithmetic-level PMA support as defined in [Volume II, Section 3.6.3.1](#).

4.13. Ziccamoc Extension for Main Memory Compare-and-Swap

If the Ziccamoc extension is implemented, then main memory regions with both the coherence and cacheability PMAs must provide AMOCASQ-level PMA support as defined in [Volume II, Section 3.6.3.1](#).

4.14. Zicclsm Extension for Main Memory Misaligned Accesses

If the Zicclsm extension is implemented, then misaligned loads and stores to main memory regions with both the coherence and cacheability PMAs must be supported.



This definition includes vector memory accesses. It does not include any instructions in the various Za extensions.*



Even though mandated, misaligned loads and stores might execute extremely slowly. Standard software distributions should assume their existence only for correctness, not for performance.

4.15. Zic64b Extension for 64-byte Cache Blocks

If the Zic64b extension is implemented, then cache blocks must be 64 bytes in size, naturally aligned in the address space.

4.16. Zimop Extension for May-Be-Operations

This chapter defines the Zimop extension, which introduces the concept of instructions that *may be operations* (MOPs). MOPs are initially defined to simply write zero to `x[rd]`, but are designed to be redefined by later extensions to perform some other action. The Zimop extension defines an encoding space